

# EKS + GitHub Actions

## Complete DevOps Project Guide

*From Zero to Live App — ELI5 Edition*

*Terraform • AWS EKS • Docker • SonarQube • Trivy • Kubernetes*

| Phase    | What You Do                                  |
|----------|----------------------------------------------|
| Phase 1  | Launch EC2 + Install Tools                   |
| Phase 2  | Write & Run Terraform — Build EKS Cluster    |
| Phase 3  | Connect kubectl to EKS                       |
| Phase 4  | EKS Auth Setup (aws-auth) — Most Tricky Part |
| Phase 5  | Create All Kubernetes Manifest Files         |
| Phase 6  | Set Up GitHub Secrets                        |
| Phase 7  | Write GitHub Actions Pipelines (CI + CD)     |
| Phase 8  | Apply Manifests Once from EC2                |
| Phase 9  | Push Code — Watch Pipelines Run              |
| Phase 10 | Verify App is Live                           |
| Mistakes | All Mistakes Made + Fixes + Alternatives     |

# What Are We Building?

## ELI5 ANALOGY

Think of it like a factory. You write code (raw material). GitHub Actions is the conveyor belt that automatically builds, tests, and packages it. Kubernetes (EKS) is the warehouse that runs your packaged app for real users. Terraform builds the warehouse from a blueprint.

## The complete flow:

You push code to GitHub

- CI Pipeline starts automatically
  - Maven builds your Java app into a .war file
  - SonarQube checks code quality
  - OWASP checks for security vulnerabilities in libraries
  - Docker packages .war into an image
  - Trivy scans the Docker image for security holes
  - Image pushed to Docker Hub
  - .war file backed up to S3
- CD Pipeline starts automatically (after CI passes)
  - kubectl connects to your EKS cluster
  - kubectl apply deploys the new image to Kubernetes
  - Your app is live with zero downtime

| Tool / Service        | What It Does (ELI5)                                |
|-----------------------|----------------------------------------------------|
| Developer writes code | Raw material                                       |
| GitHub                | Stores your code, triggers robots                  |
| GitHub Actions        | The robot that does CI/CD automatically            |
| Maven                 | Compiles Java code into a .war file                |
| SonarQube             | Code quality checker — like Grammarly for code     |
| OWASP                 | Checks if your libraries have known hacks          |
| Docker                | Packages your app into a portable container        |
| Trivy                 | Security scanner for Docker images                 |
| Docker Hub            | GitHub but for Docker images                       |
| S3 Bucket             | Backup storage for your .war artifact              |
| Terraform             | Builds AWS infrastructure from code                |
| AWS EKS               | Amazon's managed Kubernetes — runs your containers |
| kubectl               | TV remote for your Kubernetes cluster              |
| Kubernetes Manifests  | YAML files that tell K8s how to run your app       |

# Phase 1 — Launch EC2 and Install Tools

## Why this phase?

### ELI5 ANALOGY

You need a computer inside AWS to run Terraform commands. Your laptop works too, but an EC2 is always-on and already inside AWS with low latency. Think of it as your control room.

### Step 1.1 — Launch EC2

Use t2.medium or larger. Amazon Linux 2 or Ubuntu 22.04. Open port 22 (SSH) in security group.

### Step 1.2 — Install Terraform

```
sudo apt update && sudo apt install -y unzip wget
wget https://releases.hashicorp.com/terraform/1.7.0/terraform_1.7.0_linux_amd64.zip
unzip terraform_1.7.0_linux_amd64.zip
sudo mv terraform /usr/local/bin/
terraform --version
```

### ELI5 ANALOGY

Terraform is like an order form for AWS. Instead of clicking buttons in the console, you write what you want and Terraform builds it.

### Step 1.3 — Install AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip && sudo ./aws/install
aws configure
# Enter: Access Key ID, Secret Access Key, Region: us-east-1, Output: json
```

### Step 1.4 — Install kubectl

```
curl -LO https://dl.k8s.io/release/v1.32.0/bin/linux/amd64/kubectl
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version --client
```

### ELI5 ANALOGY

kubectl is the TV remote for Kubernetes. It sends commands to your cluster. Without it you cannot interact with the cluster at all.

# Phase 2 — Write and Run Terraform to Build EKS Cluster

## Why Terraform?

### ELI5 ANALOGY

Instead of spending hours clicking in the AWS console (and possibly forgetting a step), you write a blueprint in code. Terraform reads that blueprint and builds everything automatically — VPC, subnets, EKS cluster, node groups — in one command.

### Step 2.1 — Clone the Terraform code

```
git clone https://github.com/devops0014/k8s-project.git
cd k8s-project/Eks-terraform
```

### Step 2.2 — Edit two files before running

| File       | What to Change and Why                                                                                   |
|------------|----------------------------------------------------------------------------------------------------------|
| backend.tf | Change bucket name to YOUR S3 bucket. Terraform saves its state file here so it remembers what it built. |
| main.tf    | Add YOUR subnet IDs from your AWS VPC. Terraform needs to know which network to put the cluster in.      |

### Step 2.3 — Run Terraform commands in order

```
terraform init      # Downloads AWS plugins (like npm install)
terraform validate  # Checks for typos in your code
terraform plan      # Shows you WHAT it will build (preview – nothing created yet)
terraform apply --auto-approve # Actually builds everything (takes ~15 mins)
```

### What Terraform builds for you:

| What Gets Created | Why It Exists                                           |
|-------------------|---------------------------------------------------------|
| EKS Control Plane | The brain of Kubernetes — manages all your worker nodes |
| Worker Node Group | The actual machines that run your app containers        |
| VPC + Subnets     | The private network your cluster lives in               |
| IAM Roles         | Permissions for nodes to talk to AWS services           |
| Security Groups   | Firewall rules for the cluster                          |

### MISTAKE YOU MADE

**MISTAKE:** Many people forget to change the S3 bucket name in backend.tf and run terraform init — it fails immediately because that bucket doesn't exist in their account.

### THE FIX

**FIX:** Create an S3 bucket in your AWS account first, then put that exact bucket name in backend.tf before running any terraform command.

## Phase 3 — Connect kubectl to Your EKS Cluster

### Why this step?

#### ELI5 ANALOGY

After Terraform builds the cluster, your kubectl has no idea it exists. This step is like pairing your TV remote to a new TV. You run one AWS command and it writes a config file that tells kubectl exactly which cluster to talk to and how to authenticate.

```
# Connect kubectl to the cluster
aws eks update-kubeconfig --region us-east-1 --name EKS_CLOUD

# Verify it worked – should list your worker nodes
kubectl get nodes

# Should show something like:
# NAME                                STATUS    ROLES    AGE
# ip-10-0-1-x.ec2.internal            Ready     5m
```

#### ELI5 ANALOGY

The command writes a file at `~/.kube/config` — this is the TV remote pairing file. From now on every kubectl command reads this file to know which cluster to target.

## Phase 4 — EKS Auth Setup (Most Tricky Part)

### Why does this exist at all?

#### ELI5 ANALOGY

EKS has TWO separate security doors. Door 1 is AWS IAM — controls who can call AWS APIs. Door 2 is Kubernetes RBAC — controls who can do things INSIDE the cluster. Even if you pass Door 1, Door 2 will stop you unless you are on its list. The aws-auth ConfigMap IS that list for Door 2.

#### The two-door security model:

| Security Layer           | What It Controls and How to Configure It                                                                                                                                 |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Door 1 — AWS IAM         | IAM Policy attached to your user. Allows calling AWS APIs like <code>eks:DescribeCluster</code> . You handle this in AWS Console by attaching policies to your IAM user. |
| Door 2 — Kubernetes RBAC | The aws-auth ConfigMap inside the cluster. Allows doing things INSIDE Kubernetes like deploying pods. You handle this by editing the ConfigMap.                          |

#### Step 4.1 — Attach EKS permissions to your IAM user

Go to: AWS Console → IAM → Users → terraform → Add permissions → Attach policies directly

**Attach these policies (AmazonEKSFulAccess was removed by AWS — use these instead):**

| Policy Name                        | Why You Need It                                 |
|------------------------------------|-------------------------------------------------|
| AmazonEKSClusterPolicy             | Allows describing and connecting to the cluster |
| AmazonEKSWorkerNodePolicy          | Allows interacting with worker nodes            |
| AmazonEKSServicePolicy             | Allows EKS service level actions                |
| AmazonEC2ContainerRegistryReadOnly | Allows pulling Docker images                    |
| AmazonEKS_CNI_Policy               | Allows networking inside the cluster            |

#### ALTERNATIVE

ALTERNATIVE: If you cannot find the above policies, create an inline policy. Go to IAM → Users → terraform → Add permissions → Create inline policy → JSON tab. Paste the EKS permissions JSON manually. This always works regardless of what managed policies AWS has removed.

#### Step 4.2 — Add your IAM user to the aws-auth ConfigMap

#### ELI5 ANALOGY

The aws-auth ConfigMap is the guest list for your Kubernetes cluster. Right now it only has the worker nodes on the list. You need to add your terraform IAM user so GitHub Actions can deploy to the cluster.

#### Safe method — create a file and apply it (better than vim):

```
cat > /tmp/aws-auth-patch.yaml << 'EOF'
```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: aws-auth
  namespace: kube-system
data:
  mapRoles: |
    - rolearn: arn:aws:iam::YOUR-ACCOUNT-ID:role/eks-node-group-cloud
      username: system:node:{{EC2PrivateDNSName}}
      groups:
        - system:bootstrappers
        - system:nodes
  mapUsers: |
    - userarn: arn:aws:iam::YOUR-ACCOUNT-ID:user/terraform
      username: terraform
      groups:
        - system:masters
EOF

kubectl apply -f /tmp/aws-auth-patch.yaml

# Verify both mapRoles and mapUsers are present
kubectl get configmap aws-auth -n kube-system -o yaml

```

#### MISTAKE YOU MADE

MISTAKE 1: Editing aws-auth in vim and the edit not saving correctly. Vim is unforgiving — one wrong keystroke and the file is corrupted or the edit is lost.

#### THE FIX

FIX 1: Use the file method above instead of vim. Create the file with cat, then apply with kubectl apply. Much safer and repeatable.

#### MISTAKE YOU MADE

MISTAKE 2: Trying to attach AmazonEKSFULLAccess — this policy was removed by AWS and no longer exists in the console.

#### THE FIX

FIX 2: Attach the five individual EKS policies listed above, OR create an inline policy with the exact permissions JSON.

#### MISTAKE YOU MADE

MISTAKE 3: Adding mapUsers but forgetting that the mapRoles section must stay exactly as it was. If you accidentally delete or corrupt mapRoles, your worker nodes lose connection to the cluster and all pods fail.

#### THE FIX

**FIX 3:** Always copy the existing mapRoles content exactly when reapplying aws-auth. Never remove or modify the mapRoles section.



# Phase 5 — Create All Kubernetes Manifest Files

## What are manifest files?

### ELI5 ANALOGY

Manifest files are instruction sheets for Kubernetes. Just like a recipe tells a chef what to cook and how, a manifest file tells Kubernetes what to create and how to run it. They are simple YAML text files that live in your repo inside the manifests/ folder.

### Folder structure:

```
your-repo/
■■■ manifests/
    ■■■ namespace.yml      # Create the 'webapps' room
    ■■■ serviceaccount.yml # Create an identity for GitHub Actions
    ■■■ role.yml           # Define what that identity can do
    ■■■ rolebinding.yml    # Link identity to permissions
    ■■■ secret.yml         # Generate a login token
    ■■■ app-deployment.yml # Run your Java app in pods
    ■■■ db-deployment.yml  # Run your database in pods
    ■■■ service.yml        # Expose app to the internet
```

| File               | What It Creates (ELI5)                                     | Why It Must Exist                                                                    |
|--------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------|
| namespace.yml      | Creates a room called 'webapps' inside Kubernetes          | Without this room, all other files fail with 'namespace not found'                   |
| serviceaccount.yml | Creates a user account called 'jenkins' for GitHub Actions | Kubernetes needs to know WHO is deploying. This is that identity.                    |
| role.yml           | Defines what the jenkins account is allowed to do          | Having an account with zero permissions is useless. This gives it a job description. |
| rolebinding.yml    | Links the role TO the service account                      | The role and account exist separately. This file connects them together.             |
| secret.yml         | Generates a login token for the jenkins account            | GitHub Actions uses this token to authenticate with Kubernetes.                      |
| app-deployment.yml | Tells K8s to run 2 copies of your Java app                 | This is the actual instruction to run your app container.                            |
| db-deployment.yml  | Tells K8s to run your database container                   | Your app needs a database. This runs it inside the cluster.                          |
| service.yml        | Creates a public URL (Load Balancer) for your app          | Without this, your app runs but nobody outside can reach it.                         |

### namespace.yml

```
apiVersion: v1
```

```
kind: Namespace

metadata:
  name: webapps
```

### serviceaccount.yml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: webapps
```

### role.yml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: app-role
  namespace: webapps
rules:
  - apiGroups: [ "", "apps", "autoscaling", "batch",
                "extensions", "policy", "rbac.authorization.k8s.io" ]
    resources:
      - pods
      - deployments
      - services
      - configmaps
      - endpoints
      - horizontalpodautoscalers
      - replicaset
      - serviceaccounts
    verbs: [get, list, watch, create, update, patch, delete]
```

### rolebinding.yml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-rolebinding
  namespace: webapps
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: app-role
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: webapps
```

### secret.yml

```
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: mysecretname
  namespace: webapps
  annotations:
    kubernetes.io/service-account.name: jenkins
```

### app-deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  namespace: webapps
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: shaikmustafa/socialmedia:app
          ports:
```

```
- containerPort: 8080
```

### service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
  namespace: webapps
spec:
  type: LoadBalancer
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

## Phase 6 — Set Up GitHub Secrets

### Why GitHub Secrets?

#### ELI5 ANALOGY

You never put passwords or API keys directly in your pipeline code. If you did, anyone who views your repo would see them. GitHub Secrets stores them encrypted and injects them into the pipeline as `${{ secrets.NAME }}` — the actual values never appear in logs or code.

#### Where to add them:

GitHub repo → Settings → Secrets and variables → Actions → New repository secret

| Secret Name           | What It Is and Where to Get It                                                      |
|-----------------------|-------------------------------------------------------------------------------------|
| AWS_ACCESS_KEY_ID     | Your terraform IAM user access key. Allows GitHub Actions to talk to AWS.           |
| AWS_SECRET_ACCESS_KEY | Your terraform IAM user secret key. Pair with the above.                            |
| DOCKER_USERNAME       | Your Docker Hub username. Used to push images.                                      |
| DOCKER_PASSWORD       | Your Docker Hub password. Used to authenticate the push.                            |
| SONAR_TOKEN           | API token from SonarQube. Go to SonarQube → My Account → Security → Generate token. |
| SONAR_HOST_URL        | URL of your SonarQube server. Format: <code>http://YOUR-EC2-IP:9000</code>          |

# Phase 7 — GitHub Actions Pipeline (Combined Single File)

## Why one file instead of two?

### ELI5 ANALOGY

Originally we tried two separate files using `workflow_run` trigger to chain them. This is unreliable — GitHub sometimes ignores the trigger if the file was added after the first CI run. Using one file with two jobs and the `'needs:'` keyword is guaranteed to work every time.

### MISTAKE YOU MADE

MISTAKE: Using two separate workflow files (`ci-pipeline.yml` and `cd-pipeline.yml`) connected by `workflow_run` trigger. The CD pipeline never started even when CI passed.

### THE FIX

FIX: Combine into one `pipeline.yml` file. Use `'needs: build'` in the deploy job. This forces GitHub to run them sequentially in the same workflow. 100% reliable.

### ALTERNATIVE

ALTERNATIVE: Keep two files but add `'branches: [main]'` inside the `workflow_run` trigger block AND make sure the CD file existed on main before the very first CI run ever happened.

**File location:** `.github/workflows/pipeline.yml`

## Job 1 — Build (CI)

```
name: CI Pipeline

on:
  push:
    branches: [ main ]

jobs:
  build:
    name: Build and Push
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Java 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
```

```

    distribution: 'temurin'

- name: Build with Maven
  run: mvn clean package -DskipTests

- name: Copy WAR to Docker folder
  run: cp -r target Docker-app

- name: SonarQube analysis
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
    SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }
  run: mvn sonar:sonar -Dsonar.projectKey=MyProject

- name: OWASP dependency check
  uses: dependency-check/Dependency-Check_Action@main
  with:
    project: 'MyProject'
    path: '.'
    format: 'HTML'

# NOTE: Do NOT add env: JAVA_HOME here – causes Docker container conflict

- name: Build Docker images
  run: |
    docker build -t appimage Docker-app
    docker build -t dbimage Docker-db

- name: Trivy scan
  run: |
    curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s -- -b /usr/local/bin
    trivy image appimage
    trivy image dbimage

- name: Push to Docker Hub
  env:
    DOCKER_USERNAME: ${ secrets.DOCKER_USERNAME }
    DOCKER_PASSWORD: ${ secrets.DOCKER_PASSWORD }
  run: |
    echo "$DOCKER_PASSWORD" | docker login -u "$DOCKER_USERNAME" --password-stdin
    docker tag appimage $DOCKER_USERNAME/socialmedia:app
    docker tag dbimage $DOCKER_USERNAME/socialmedia:db

```

```

    docker push $DOCKER_USERNAME/socialmedia:app
    docker push $DOCKER_USERNAME/socialmedia:db

- name: Upload artifact to S3

env:

    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
    AWS_DEFAULT_REGION: us-east-1

run: |

    aws s3 cp target/vprofile-v2.war s3://YOUR-BUCKET-NAME/vprofile-v2.war

```

## Job 2 — Deploy (CD)

```

deploy:

    name: Deploy to EKS
    runs-on: ubuntu-latest
    needs: build          # Waits for build job to pass first
                        # If build fails this job never starts

    steps:

    - name: Checkout code
      uses: actions/checkout@v4
      # Needed – each job runs on a fresh machine
      # Code from build job is NOT available here

    - name: Install kubectl
      run: |
        curl -LO https://dl.k8s.io/release/v1.32.0/bin/linux/amd64/kubectl
        chmod +x kubectl
        sudo mv kubectl /usr/local/bin/
        kubectl version --client

      # NOTE: Must install again – build job's machine was destroyed
      # Each job = brand new blank machine

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
        aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
        aws-region: us-east-1

    - name: Connect kubectl to EKS

```



```
run: aws eks update-kubeconfig --region us-east-1 --name EKS_CLOUD

- name: Deploy to Kubernetes
  run: kubectl apply -f manifests/ --validate=false
  # --validate=false skips openapi credential check
  # Safe to use when aws-auth is correctly configured

- name: Verify deployment
  run: kubectl get all -n webapps
```

### MISTAKE YOU MADE

**MISTAKE:** Not installing kubectl in the deploy job. Each GitHub Actions job runs on a completely separate brand-new machine. kubectl installed in Job 1 does NOT carry over to Job 2.

### THE FIX

**FIX:** Add the kubectl install step at the start of EVERY job that uses kubectl. Same for Java, Trivy, or any other tool.

| What carries over between jobs            | What does NOT carry over                  |
|-------------------------------------------|-------------------------------------------|
| GitHub Secrets (injected by GitHub)       | Installed software (kubectl, java, trivy) |
| Docker Hub images (they're on Docker Hub) | Files you created (.war, built images)    |
| S3 files (they're on S3)                  | Your repo code (must checkout again)      |
| aws-auth configmap (it's in Kubernetes)   | Environment variables                     |

## Phase 8 — Apply Manifests Once from EC2

### Why manually the first time?

#### ELI5 ANALOGY

Your CD pipeline deploys with 'kubectl apply -f manifests/ -n webapps' but the namespace 'webapps' doesn't exist yet! You need to create it once manually before the first pipeline run. After this one-time setup, everything is automatic forever.

```
# Run these on your EC2 in order

# Step 1: Make sure kubectl is connected
kubectl get nodes

# Step 2: Apply manifest files
kubectl apply -f manifests/namespace.yml
kubectl apply -f manifests/serviceaccount.yml
kubectl apply -f manifests/role.yml
kubectl apply -f manifests/rolebinding.yml
kubectl apply -f manifests/secret.yml

# Step 3: Verify everything was created
kubectl get all -n webapps
kubectl get serviceaccount -n webapps
kubectl get role -n webapps
kubectl get rolebinding -n webapps
kubectl get secret mysecretname -n webapps
```

#### ELI5 ANALOGY

After this one-time step, your CD pipeline handles the rest forever. When it runs 'kubectl apply -f manifests/', Kubernetes sees the namespace and RBAC already exist and skips them. It only updates the deployment with the new Docker image.

# Phase 9 and 10 — Push Code and Verify App is Live

## Phase 9 — Trigger the pipeline

```
git add .
git commit -m 'add github actions pipeline and manifests'
git push origin main

# Go to GitHub → Actions tab
# Watch CI Pipeline run first
# Then deploy job starts automatically after CI passes
```

### What you see in the Actions tab:

| What You See       | What Is Happening                                                                        |
|--------------------|------------------------------------------------------------------------------------------|
| build job running  | Checkout → Maven → SonarQube → OWASP → Docker → Trivy → Push → S3                        |
| build job passed   | Green checkmark. deploy job starts automatically.                                        |
| deploy job running | Checkout → kubectl install → AWS credentials → connect to EKS → apply manifests → verify |
| deploy job passed  | Your app is live!                                                                        |

## Phase 10 — Verify your app is live

```
# SSH into EC2 and run:

# Check all pods are running
kubectl get pods -n webapps

# Should show:

# NAME                                READY   STATUS    RESTARTS
# myapp-deployment-xxx-yyy            1/1     Running   0
# myapp-deployment-xxx-zzz            1/1     Running   0

# Get the public URL
kubectl get service -n webapps

# Should show EXTERNAL-IP like:
# abc123.us-east-1.elb.amazonaws.com

# Open that URL in your browser — your app is live!
```

# Complete Mistakes Reference — Everything That Went Wrong and Why

## 1. CD pipeline never triggered

### MISTAKE YOU MADE

CAUSE: Used two separate workflow files with `workflow_run` trigger. `workflow_run` only works if the CD file existed on main before the first CI run ever happened. Since CI ran first, GitHub cached the workflow list and never picked up CD.

### THE FIX

FIX: Combine into one `pipeline.yml` file with two jobs. Use `'needs: build'` to chain them. This is handled inside the same file — GitHub cannot ignore it.

### ALTERNATIVE

ALTERNATIVE: Keep two files but add `'branches: [main]'` to `workflow_run`. Also ensure CD file was on main from day one before any commit triggered CI.

## 2. OWASP step failed with `JAVA_HOME` not defined

### MISTAKE YOU MADE

CAUSE: Added `'env: JAVA_HOME: /opt/jdk'` to the OWASP step. OWASP runs inside its own Docker container which cannot see the Java installed on the GitHub Actions machine. The path `/opt/jdk` does not exist inside that container.

### THE FIX

FIX: Remove the `env: JAVA_HOME` block entirely from the OWASP step. The action manages its own Java internally.

### ALTERNATIVE

ALTERNATIVE: Use the Maven plugin instead: `run: mvn org.owasp:dependency-check-maven:check`. No Docker container, no Java path issues.

## 3. `kubectl` not found in deploy job

### MISTAKE YOU MADE

CAUSE: Assumed `kubectl` installed in build job would be available in deploy job. Each GitHub Actions job runs on a completely separate brand-new machine. When build job finishes, that machine is destroyed. Deploy job gets a blank machine.

### THE FIX

FIX: Add the `kubectl` install step at the top of the deploy job. Must reinstall on every job that uses it.

### ALTERNATIVE

ALTERNATIVE: Use a GitHub Actions marketplace action: `uses: azure/setup-kubectl@v3`. Cleaner one-liner instead of manual `curl` install.

## 4. EKS auth failing — server asked for credentials

#### MISTAKE YOU MADE

CAUSE: The aws-auth ConfigMap only had mapRoles (worker nodes). The terraform IAM user was not added to mapUsers. Kubernetes saw the IAM user as a complete stranger and rejected every kubectl command.

#### THE FIX

FIX: Add mapUsers section to aws-auth ConfigMap with the terraform IAM user ARN and grant system:masters group. Use the file method (cat + kubectl apply) instead of vim.

#### ALTERNATIVE

ALTERNATIVE: Use eksctl to manage aws-auth: `eksctl create iamidentitymapping --cluster EKS_CLOUD --arn arn:aws:iam::ACCOUNT:user/terraform --group system:masters --username terraform`

## 5. AmazonEKSFullAccess policy not found

#### MISTAKE YOU MADE

CAUSE: AWS removed the AmazonEKSFullAccess managed policy. Searching for it in the console returns nothing. Cannot attach a policy that does not exist.

#### THE FIX

FIX: Attach five individual policies: AmazonEKSClusterPolicy, AmazonEKSWorkerNodePolicy, AmazonEKSServicePolicy, AmazonEC2ContainerRegistryReadOnly, AmazonEKS\_CNI\_Policy.

#### ALTERNATIVE

ALTERNATIVE: Create an inline policy with a custom JSON listing only the specific EKS actions needed. More secure than broad managed policies.

## 6. aws-auth vim edit not saved

#### MISTAKE YOU MADE

CAUSE: Edited aws-auth directly in vim with 'kubectl edit configmap aws-auth'. Vim is difficult to use correctly. The edit appeared to save but mapUsers was not present when verified.

#### THE FIX

FIX: Use the file method: write a complete aws-auth yaml file with cat heredoc, then apply with kubectl apply -f. Completely avoids vim.

#### ALTERNATIVE

ALTERNATIVE: Use kubectl patch: `kubectl patch configmap aws-auth -n kube-system --patch` with a JSON patch string. No editor involved.

## 7. CD deploy failing even after auth was fixed

#### MISTAKE YOU MADE

CAUSE: Running 'kubectl apply -f manifests/ -n webapps' when the namespace did not exist yet. The -n webapps flag forces all resources into that namespace, but if the namespace YAML hasn't been applied yet, it fails.

#### THE FIX

FIX: Apply the namespace.yml manually once from EC2 before running the pipeline. Then remove -n webapps from the pipeline command and let the namespace field in each YAML file handle it.

#### ALTERNATIVE

ALTERNATIVE: Add `--validate=false` to the `kubectl apply` command. This skips the openapi validation step that was also triggering credential errors.

# Quick Reference Cheatsheet

## Most used kubectl commands:

```
kubectl get nodes                # List worker nodes
kubectl get pods -n webapps      # List pods in webapps namespace
kubectl get service -n webapps   # Get public URL of your app
kubectl get all -n webapps       # Show everything in namespace
kubectl logs POD-NAME -n webapps # See app logs
kubectl describe pod POD-NAME -n webapps # Debug a pod
kubectl get configmap aws-auth -n kube-system -o yaml # Check aws-auth
```

## Most used Terraform commands:

```
terraform init    # Download plugins (run once)
terraform plan    # Preview what will be built
terraform apply --auto-approve # Build everything
terraform destroy --auto-approve # Delete everything (saves AWS costs!)
```

## Debug GitHub Actions pipeline:

```
# Add this step before any failing step to see the environment:
- name: Debug
  run: |
    echo '=== AWS Identity ==='
    aws sts get-caller-identity
    echo '=== kubectl nodes ==='
    kubectl get nodes
    echo '=== namespaces ==='
    kubectl get namespaces
```

## What the .war file is and what goes where:

| What                | Explanation                                                                    |
|---------------------|--------------------------------------------------------------------------------|
| Maven produces      | target/vprofile-v2.war — your whole Java app in one file                       |
| .war goes to S3     | Backup copy. Use if you ever want to deploy directly to Tomcat without Docker. |
| .war goes to Docker | Dockerfile copies it into an image. Docker Hub hosts that image.               |
| Kubernetes uses     | The Docker image from Docker Hub — NOT the .war file directly                  |

## Every time you push code after setup — what happens automatically:

```
git push origin main
```

→ build job: Maven → SonarQube → OWASP → Docker → Trivy → DockerHub → S3

→ deploy job: kubectl apply → Kubernetes rolling update → zero downtime

→ your app is updated in ~5-10 minutes with no manual work